ZigZag: A Memory-Centric Rapid DNN Accelerator Design Space Exploration Framework

Linyan Mei†, Pouya Houshmand†, Vikram Jain, Sebastian Giraldo, and Marian Verhelst

MICAS, ESAT, KU Leuven

†These authors contributed equally to this work.

***Abstract*—Building efficient embedded deep learning systems requires a tight co-design between DNN algorithms, memory hierarchy, and dataflow. However, owing to the large degrees of freedom in the design space, finding an optimal solution through the implementation of individual design points becomes infeasible. Recently, several estimation frameworks for fast design space exploration (DSE) have emerged, yet they either suffer from long runtimes or a limited exploration space. This work introduces ZigZag, a memory-centric rapid DNN accelerator DSE framework which extends the DSE with uneven mapping opportunities, in which operands at shared memory levels are no longer bound to use the same memory levels for each loop index. For this, ZigZag uses a memory-centric nested-for-loop format as a uniform representation to integrate algorithm, accelerator, and algorithm-to-accelerator mapping, and consists of three key components: 1) a latency-enhanced analytical Hardware Cost Estimator, 2) a Temporal Mapping Generator that supports even/uneven scheduling on any type of memory hierarchy, and 3) an Architecture Generator that explores the whole memory hierarchy design space. Benchmarking experiments against existing frameworks, together with three case studies at different design abstraction levels show the strength of ZigZag. Up to 33% more energy-efficient solutions are found by introducing ZigZag's uneven scheduling opportunities.**

***Index Terms*—Deep neural networks, accelerator, cost model, dataflow, mapping, scheduling, design space exploration.**

***Source code*—Source code is available at https://github.com/ZigZag-Project/zigzag.**

I. Introduction

Over the last decade, deep neural networks (DNNs) have established themselves as the principal algorithm for pattern recognition and data mining tasks, dominating the field of artificial intelligence (AI). Recent DNN models achieve greatly improved accuracies at the expense of increased depth and complexity. Executing these complex models on embedded systems becomes challenging due to resource and power constraints in edge devices.

To meet the constraints in these devices, a lot of research in the recent past, in both industry and academia, has been done towards developing specialized hardware accelerators [1]–[11] for energy-efficient and high-throughput mapping of DNN workloads. Each accelerator is designed with a different memory hierarchy and a different choice of dataflow. However, most of them are ad-hoc and local optimal designs resulting from exploration of a limited design space. It is hard to say if the configuration selected by the accelerators is the best one, given the vast design space available. Therefore it is essential

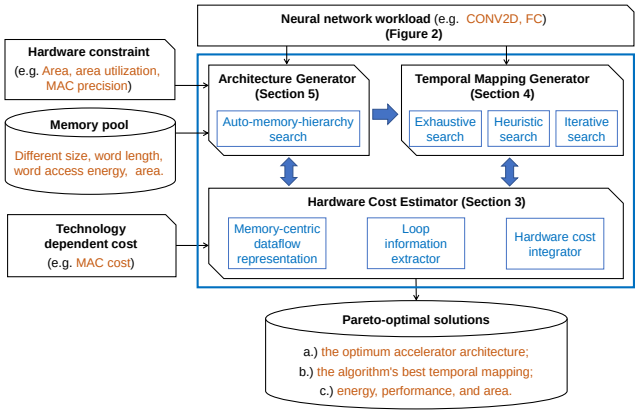


Fig. 1. ZigZag framework diagram.

to have a framework that can rapidly explore the available design space to guide designers in finding the Pareto optimal architectures with the optimal dataflow while taking in the hardware constraints and algorithmic workloads.

Many frameworks have thus emerged over the last few years targeting such hardware-software co-optimization by exploring the large design space available in the DNN system. Recent works on DSE framework in literature include Interstellar [12], SMAUG [13], Accelergy [14], Dory [15], Timeloop [16], dMazeRunner [17], MAESTRO [18], and MAGnet [19]. In order to provide a clear view in this domain, Table I compares different DSE frameworks' design space, search engine, and cost estimation strategy.

Firstly, there are three main design spaces that need to be considered in the DSE, which are the algorithm space, hardware space, and algorithm-to-hardware mapping space. Most DNN-accelerator-DSE frameworks focus on exploring the latter two. Concerning the hardware design space, some SotA frameworks support a fully flexible hardware configuration (PE array and memory hierarchy), like Timeloop [16]; while others pre-define a hardware template with certain tunable parameters, like MAGnet [19]; a last group makes specific assumptions, such as the sharing of all memory levels for the operands I/W/O and only explore within these constraints, like Insterstellar [12]. Concerning the temporal mapping space (scheduling space), all of the SotA frameworks only support even mappings, i.e. different operands need to follow the same loop blocking scheme at each shared memory level. Even and

TABLE I
DNN ACCELERATOR DSE FRAMEWORK COMPARISON

Framework	Hardware design space	Temporal mapping space	Temporal mapping generator	Cost estimation
Timeloop+Accelergy [14], [16]	Fully flexible	Even mappings	Constraint-driven	Highly fine-grained analy. model
MAESTRO [18]	Lowly flexible HW template	Even mappings	Predefined IS/WS/OS/RS	Coarse-grained analytical model
Interstellar [12]	All mem. levels shared	Even mappings	Fully flexible	Coarse-grained analytical model
dMazeRunner [17]	All mem. levels shared	Even mappings	Constraint-driven	Coarse-grained analytical model
MAGnet [19]	Highly flexible HW template	Even mappings	Constraint-driven	Real hardware implementation
Dory [15]	Fixed architecture	Even mappings	Optimization-based	Coarse-grained analytical model
SMAUG [13]	Fixed architecture	Even mappings	Fixed mapping	Cycle-accurate estimator
Ours: ZigZag	Fully flexible	Even and uneven mappings	Fully flexible with optional constraints	Fine-grained analytical model

uneven mapping/loop blocking will be discussed in more detail further in this paper.

Secondly, the DSE tools typically encompass a temporal mapping generator, a.k.a. auto-scheduler or mapper, to find the optimum (in energy or/and performance) temporal mapping scheme for mapping a certain neural network layer onto a certain accelerator architecture. Most of the DSE frameworks perform a constraint-driven search to narrow down the space and speed up the searching procedure, like deMazeRunner [17]; some formulate the scheduling process into a linear problem and utilize optimization tools to solve it, like Dory [15]; the other use predefined dataflow to generate valid mapping points, like MAESTRO [18]. Commonly used scheduling constraints and strategies include setting threshold for memory/PE array utilization and data reuse factor, and putting optimization goal for minimize certain cost functions, like the DRAM access or the overall memory traffic.

Finally, the last column in Table 1 listed the cost estimating approach adopted by each framework, in which three main categories can be noticed, 1) slow but very accurate hardware implementation based on High-Level Synthesis (HLS) [19], 2) medium-speed and accurate cycle-accurate system simulator [13], and 3) fast and generally accurate analytical model. Moreover, there are different granularity levels for analytical model [14]. Models, which distinguish memory writing from reading, consider memory word-width's impact on access cost, and take data pattern/stationarity into account in unit cost, are referred to as fine-grained models.

This work proposes ZigZag, a memory-centric rapid DNN accelerator DSE framework (Section 2). Zigzag innovates on broadening the architecture and scheduling searching space, especially on enabling fully-flexible memory hierarchies search and even/uneven auto-scheduling, and thus discovers better design points than other frameworks. ZigZag can also estimate performance (latency/throughput/MAC array utilization), important metrics of an accelerator that are lacking in some of the other frameworks. ZigZag estimates performance not only based on spatial mapping but also memory bandwidth and computing capacity. Moreover, our framework uses smarter searching strategies to explore the enlarged design space, reducing the runtime while still locating the global optimum design point as exhaustive search does. To sum up, ZigZag made the following three key contributions.

Firstly, the **memory-centric dataflow representation (Section 3)**, based on an enhanced nested-for-loop format, is proposed as a uniform representation for each design point. It integrates the information of algorithm, accelerator, and algorithm-to-accelerator spatial & temporal mapping (a.k.a. dataflow). This newly-proposed representation opens up a whole new space for DSE by decoupling the operands (W/I/O), the memory hierarchy, and the mapping scenarios. Combined with a proposed *loop relevance principle*, the framework extracts in a systematic and insightful way the key information like number of memory accesses, required memory bandwidth, etc., to derive the system's energy and performance.

Secondly, the **Temporal Mapping Generator (Section 4)** is built to generate valid temporal mapping points for any type of memory hierarchy, in which each memory level for each operand can be shared or separated, with or without spatial unrolling, under even or uneven blocking. Additionally, to cope with the enlarged design space, three fast search strategies are proposed on top of the original exhaustive search : data-stationarity-based *heuristic search*, data-reuse-based *heuristic search* and early-cost-evaluation-based *iterative search*.

Thirdly, an **Architecture Generator (Section 5)** is built on the top to construct different DNN accelerator architectures, especially focusing on the auto-generation of all valid memory hierarchies under given area budget.

Framework validation (Section 6) and three case studies (Section 7) at different design abstraction levels are conducted to assess the accuracy of the Hardware Cost Estimator, to show the strength of the uneven-mapping supportive Temporal Mapping Generator, as well as to gain insight in the vast design space,utilizing the fully-flexible Architecture Generator.

II. ZIGZAG FRAMEWORK OVERVIEW

Weight, Input, and Output are the three main operands in each DNN layer. The memory hierarchy and the data mapping scheme (dataflow) are responsible to get Weights and Inputs as efficiently as possible into the multiply-accumulate (MAC) units and collect the resulting Outputs, while maximizing data reuse in local storage.

However, the many degrees of freedom involved in designing a memory hierarchy and dataflow makes finding the optimal solution a difficult task: 1) Weight, Input, and Output can have the same or different memory organization, 2) for each memory level, Weight, Input, and Output can be stored shared or separately, 3) spatial unrolling can be applied at different memory levels (e.g. register file can be spatially unrolled into each processing element), and 4) for each memory hierarchy,

Workload	Batch size	O channel	I channel	O row	O column	W row	W column
Conv2D	B	K	C	OY	OX	FY	FX
Conv1D	B	K	C	1	OX	1	FX
Depthwise Conv*	B	1	1	OY	OX	FY	FX
Pointwise Conv	B	K	C	OY	OX	1	1
Matrix-Vector Multi (FC)	1	K	C	1	1	1	1
Matrix-Matrix Multi	B	K	C	1	1	1	1

* Repeat 'C' times to finish the whole Depthwise Conv (C = K in Depthwise conv layer).

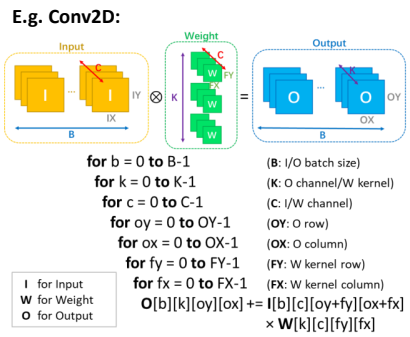


Fig. 2. Commonly used neural network workload summary.

hundreds of thousands of possible schedules exist that strongly impact energy and latency. For these reasons, an automatic tool which can rapidly explore the vast design space becomes a necessity.

ZigZag targets the automatic exploration of all valid memory hierarchies, given an area constraint, workload size information, and a memory pool. This requires an extension of the SotA frameworks on several aspects.

ZigZag contains three key components: 1) an enhanced analytical Hardware Cost Estimator, 2) an efficient and flexible Temporal Mapping Generator, and 3) a memory-centric Architecture Generator. Together, they discover Pareto-optimal design points, each including accelerator architecture, the algorithm's best schedule, and corresponding hardware cost, as shown in Figure 1.

Besides working in the full-function mode, in which the design space of both architecture and schedule are fully explored, ZigZag can also work in several partial-function modes, in which architecture and/or schedule can be partially or fully pre-defined. For example, if a designer wants to constrain stationarity in the inner-PE to output stationarity to assess the impact of the other factors for this specific architecture, it is possible to freeze degrees of freedom of the DSE and only open the upper levels to the tool to explore. This will be illustrated in several case studies in Section VII.

III. Hardware Cost Estimator

The ZigZag Hardware Cost Estimator targets the estimation of energy and performance (PE array utilization, throughput, and latency) given a certain workload size, dataflow (temporal and spatial mappings), and memory hierarchy.

It innovates on 1) Memory-centric dataflow representation (Section III-A) to capture the interaction between dataflow and memory hierarchy; 2) a loop relevance principle (Section III-B), to extract basic technology-independent hardware and data attributes from loop sets, such as memory access count and required memory bandwidth; 3) a technology- and memory-bandwidth-aware hardware cost integrator (Section III-C), capable of not only extracting energy, but also latency.

A. Memory-Centric Dataflow Representation

A uniform and concise data representation format lays the foundation for the exploration of the enlarged design space and is required to support all forms of memory sharing, loop blocking (loop tiling), loop ordering, and spatial unrolling for each operand (W/I/O) at each memory level. The proposed *Memory-Centric Dataflow Representation* well captures all memory hierarchy attributes as well as spatial and temporal algorithm-to-hardware mapping schemes.

Figure 3 illustrates the proposed memory-centric dataflow representation using the same loop name notation as Figure 2. The depicted dataflow is the energy-optimal dataflow found by ZigZag out of hundreds of thousands of possible dataflows for mapping AlexNet convolutional layer 2 (with B=1) onto Eyeriss V1 architecture [2][1], leading to 20% energy savings

a) Loop format

loop index	Weight			Input			Output		
11	DRAM	K	32		K	32	DRAM	K	32
10		C	12	Global	C	12	Global	C	12
9	FYu|OYu|OYu	5|13|2		Buffer			Buffer		
8		OX	13		OX	13		OX	13
7							FYu|OYu|OYu	5|13|2	
6		C	2		C	2		C	2
5		OX	2		OX	2		OX	2
4				FYu|OYu|OYu	5|13|2				
3		FX	5		FX	5		FX	5
2	Inner-PE	C	2	Inner-PE	C	2	Inner-PE	C	2
1	Reg File	K	8	Reg File	K	8	Reg File	K	8
0	MAC Level			MAC Level			MAC Level		

b) Code format

```
Temporal mapping: {'W': [[], [(K, 8), (C, 2), (FX, 5), (OX, 2), (C, 2), (OX, 13)], [(C, 12), (K, 32)]],
                   'I': [[], [(K, 8), (C, 2), (FX, 5)], [(OX, 2), (C, 2), (OX, 13), (C, 12), (K, 32)], []],
                   'O': [[], [(K, 8), (C, 2), (FX, 5), (OX, 2), (C, 2)], [(OX, 13), (C, 12)], [(K, 32)]]}

Spatial mapping  : {'W': [[], [(FY, 5), (OY, 13), (OY, 2)], []],
                    'I': [[], [(FY, 5), (OY, 13), (OY, 2)], [], []],
                    'O': [[], [(FY, 5), (OY, 13), (OY, 2)], [], []]}
// Each [] indicates one architecture level.
// A list of [] from left to right are MAC level up to DRAM level.

Memory sharing:  {[('I', 1), ('O', 1)],
                  [('W', 1), ('I', 2), ('O', 2)]}
// Input's 1st level memory and Output's 1st level memory share Global Buffer.
// Weight's 1st level memory, Input's 2nd level memory, and Output's 2nd level memory
share DRAM.

Memory size (bit): {'W': [3584, 16777216],
                    'I': [192, 884736, 16777216],
                    'O': [384, 884736, 16777216]}
// Memory size at 0th , 1st (, 2nd) level of memory for W, I, O.

Memory bandwidth, Memory type, Memory word access cost, Memory area,
Operand precision (Weight, Input, Partial Output, and Final Output), …
```

Fig. 3. An example of memory-centric dataflow representation (it is the best dataflow out of hundreds of thousands possible dataflows of mapping AlexNet convolutional layer 2 onto Eyeriss V1 architecture [2]).

Figure 3 illustrates the proposed memory-centric dataflow representation using the same loop name notation as Figure 2. The depicted dataflow is the energy-optimal dataflow found by ZigZag out of hundreds of thousands of possible dataflows for mapping AlexNet convolutional layer 2 (with B=1) onto Eyeriss V1 architecture [2][1], leading to 20% energy savings

[1]In this paper, We adopt the AlexNet layer dimensions reported in Eyriss paper [2] for fair comparisons, which are different to the layer dimensions reported in AlexNet paper [20].

TABLE II
LOOP RELEVANCE'S IMPLICATION ON DATAFLOW

Loop Impact	✔ r loop			✘ ir loop			? pr loop pairs		
	Spatial	Spatio-temporal	Temporal	Spatial	Spatio-temporal	Temporal	Spatial	Spatio-temporal	Temporal
W	Unicast	Unicast	Fetch new Weight	Broadcast	Propagate systolically	Keep stationary			
I	Unicast	Unicast	Fetch new Input	Broadcast	Propagate systolically	Keep stationary	Broadcast diagonally	"FIFO Effect"	"FIFO Effect"
O	Uni-collect	Uni-collect	Generate new Output	Sum up spatially	Accumulate systolically	Accumulate stationarily			

compared to the original dataflow used. Notice that in this example, the memory hierarchy and spatial unrolling settings are the same as Eyeriss, while only the temporal mapping is different, i.e. different loop blocking and loop ordering. This example will be used throughout this paper to explain various aspects of the framework.

In Figure 3a, the representation defines, from left to right, the dataflow information of the three operands separately, using three sets of nested for-loops. Inside each set, the architectural levels are represented from bottom to top (divided by bold lines), starting from the MAC units, over potential register file and/or SRAM (Global Buffer) levels, all the way up to DRAM. For each operand, each alphanumeric pair indicates a for-loop, e.g. the first term "`K 32`" is equivalent to "`for k = 0 to 32-1`". Assigning these for-loops into different architectural level is loop blocking (loop tiling) and fixing the order of all the for-loops inside one level is loop reordering. The "`u`" suffix after a loop name indicates spatial unrolling, such as "`FYu`". The format "`Au|Bu`" is inherited form [12], meaning that both the A and B loop dimensions are spatially unrolled. In Figure 3b, more detailed information is given, in which temporal mapping (schedule), spatial mapping, memory sharing, memory size, etc. are specified for each operand at each architectural level.

By combining Figure 3a with 3b, three key attributes of this representation can be observed: 1) not all of the operands have the same number of memory levels, e.g. in this example Weight has two memory levels while Input and Output have three; 2) not all of the operands that have the same memory level share physical memory, e.g. the Inner-PE Register File of W/I/O are separated; 3) not all of the operands that share a physical memory have the same/even loop blocking, e.g. Input and Output share the Global Buffer, but with a different loop blocking boundary.

Furthermore, it can be noted that temporal loops of all operands should follow the same order to maintain functional equivalence, while spatial loops can be relocated. This is because spatial mapping in this representation indicates which loop dimension is unrolled at which architecture level and to what extent, which is fully configurable in ZigZag. In this particular example, following the Eyeriss settings, W, I, and O have the same spatial mapping at the same memory level, i.e. "`FYu|OYu|OYu 5|13|2`" at Register File level.

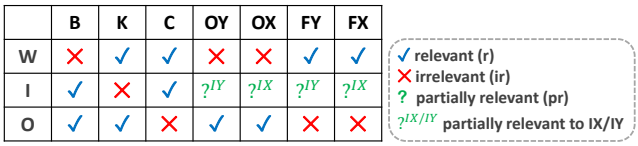


Fig. 4. Loop type categorized by relevance.

B. Loop Information Extractor Based On the Loop Relevance Principle

The enhanced representation from Section 3.1 will now be combined with a loop relevance principle, to systematically analyze and extract basic technology-independent hardware and data attributes.

Convolutional layers are based on a 7D computing space with three 4D operands: Weight, Input, and Output; which implies not all 7 dimensions are relevant to each operand. Figure 4 shows the loop relevance principle, in which all 7 loop dimensions are categorized as relevant (r), irrelevant (ir), or partially relevant (pr) to each operand. For Weight and Output, this is straightforward since all 7 computing space dimensions are either parallel (relevant) or orthogonal (irrelevant) to their own 4D data space. Looping through those 'r' loops indicates new data need to be fetched or generated, while looping through those 'ir' loops creates various data reuse opportunities, as shown in Table II.

Input, however, also has 'pr' loops besides the 'r' and 'ir' loops. As presented in the right example of Figure 2, Input's dimensions `IX` and `IY` do not show up in the convolution formula directly, instead they are indirectly present through `OX` and `FX` (for `IX`); `OY` and `FY` (for `IY`). As such, `OX`, `FX`, `OY`, `FY` are denoted as partially relevant (pr) loops for Input. `OX`, `FX` (resp. `OY` and `FY`) form a 'pr' loop pair. For a 'pr' loop pair, data reuse opportunities arise when the sum of their indices remains constant while the computation is looping through its space.

Figure 5 provides a summary of pr-loop-pair-triggered input data reuse. Such 'pr' creates alternative data reuse opportunities for spatially, temporally or spatio-temporally unrolled loops. For spatial unrolling, inputs can be broadcasted diagonally in a PE array, as done in Eyeriss [2], where `FY` and `OY` are spatially unrolled onto the 2D PE array, allowing a diagonal broadcast of inputs. For temporal and spatio-temporal unrollings, data reuse is possible through a FIFO buffer which shifts the input data over consecutive clock cycles. An example of this can be found in Envision [1], where `OX` is spatially

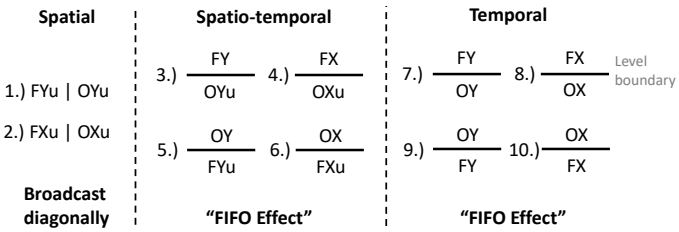


Fig. 5. Partial-relevant (pr) loop patterns that trigger special Input data reuse.

unrolled and FX is the innermost temporal loop on top, same as Figure 5 (4), making the sum of FX and OX a constant in neighboring PE locations across consecutive cycles, enabling to reuse Inputs in a FIFO manner.

The benefit of this loop relevance principle is the simplification and unification of the procedure for extracting key information from the W/I/O loops sets towards estimating system energy and performance. To show the key ideas of this procedure, an equation summary is provided in Table III and a detailed demonstration is given in Figure 6, in which the Output loop set is analyzed (the same/similar procedure would be repeated for Weight/Input, not shown).

1.) Data Size in individual memory unit at current level can be derived by multiplying together the dimensionality of all the 'r' loops at the current level and all levels below, together with all 'ru' loops (spatially unrolled 'r' loop) at all levels below. This can be seen in the first line of Table III, in which Li means current memory level, $L(i-1)$ means one level below the current memory level, and $Lmin$ means the lowest memory level. Let us apply this to a specific example, given in Figure 6. The required Output data storage inside each PE (16) is calculated by multiplying the dimensionalty of Level-1 'r' loops 1 and 5; the Data Size of the Output inside of Global Buffer (5408) is calculated by multiplying the dimensionality of Level-1,2 'r' loops (1, 5, 8) and Level-1 'ru' loops (7.2, 7.3). Later for other metrics calculation, readers can always refer to the practical case in Figure 6 for validation.

2.) Data Size in total at current level can be easily calculated by multiplying the individual Data Size in each memory unit with the dimensionality of all 'ru' loops at current level. Notice that the unit of the Data Size is number of elements. In order to obtain number of bits, the precision of the operands needs to be considered. Generally speaking, partial outputs have a higher precision than weights, inputs, and final outputs. The ability to distinguish partial outputs from final outputs is critical in the framework for accurate hardware cost estimation. ZigZag can easily handle through its 'r' vs 'ir' loop representation. The final output is generated at the level of the uppermost 'ir' loop., e.g. the L2 Global Buffer in Figure 6. As such, the data traffic between L2 and L3 is unidirectional.

3.) Number of MAC Operation supported by current level Data Size is calculated by multiplying together all the loops' dimensionalty ('r', 'ir', 'ru', and 'iru') from the lowest level up to the current level.

4.) Turnaround cycles are number of cycles certain memory can keep operating with the data it contains, which is an important metrics for later required memory bandwidth computation. It can be calculated by multiplying together all the temporal loops' dimensionalty ('r' and 'ir') from the lowest level up to the current level.

TABLE III
EQUATIONS FOR LOOP INFORMATION EXTRACTION

Metrics	Comment	Equation
Data Size @ Level i	Data Size in individual unit	$\prod_{Lmin}^{Li} r \cdot \prod_{Lmin}^{L(i-1)} ru$
	Data Size in total	$\prod_{Lmin}^{Li} r \cdot \prod_{Lmin}^{Li} ru$
MAC Operation @ Level i	Supported by its Data Size	$\prod_{Lmin}^{Li} r \cdot \prod_{Lmin}^{Li} ru \cdot \prod_{Lmin}^{Li} ir \cdot \prod_{Lmin}^{Li} iru$
Turnaround cycles @ Level i	Supported by its Data Size	$\prod_{Lmin}^{Li} r \cdot \prod_{Lmin}^{Li} ir$
Data reuse factor @ Level i	Total data reuse factor	$\prod_{Li} ir \cdot \prod_{Li} iru$
	Temporal data reuse factor	$\prod_{Li} ir$
	Spatial data reuse factor	$\prod_{Li} iru$
Unit count @ Level i	Total unit count	$\prod_{Li}^{Lmax} ru \cdot \prod_{Li}^{Lmax} iru$
	Duplicate unit count	$\prod_{Li}^{Lmax} iru$
	Unique unit count	$\prod_{Li}^{Lmax} ru$
Memory access count @ Level i	One-way for I/W; possibly two-way for O because of partial output.	$\textbf{Total Operand Size} \cdot \prod_{Li}^{Lmax} \textbf{Total data reuse factor}$
Required memory bandwidth @ Level i (write bandwidth for W/I, read bandwidth for O)	With double-buffering	$\frac{\textbf{Total Data Size @ Level i}}{\textbf{Turnaround cycles @ Level i}}$
	Without double-buffering	$\frac{\textbf{Total Data Size @ Level i}}{\textbf{Turnaround cycles @ Level i}} \cdot \prod_{Li} ir\_top$

5.) Total data reuse factor at current level is the product of all the irrelevant loops' dimensionalty ('ir' and 'iru') at current level. The product of only 'ir' loops is the temporal data reuse factor, while the product of only 'iru' loops is the spatial data reuse factor.

6.) Total unit count is a metrics that measures how many hardware components are at certain level, which is only related to spatial unrolled loops. Total unit count at current level is the product of all the spatial loops' dimensionalty ('ru' and 'iru') from the current level up to the highest level.

7.) Duplicate unit count is a measurement of how many hardware components at certain architectural level that contain same data, which is captured by the product of all the irrelevant spatial loops' dimensionalty ('iru') from the current level up to the highest level.

8.) Unique unit count is similar to Duplicate unit count, but for counting how many hardware components that contain different data, thus all the relevant spatial loops' dimensionalty ('ru') are timed together from the current to highest level.

9.) Memory access count, as the core metrics for later mem-

AlexNet Conv2 basic information											
Total MAC operations: **207667200**			Active MAC unit: 130			Ideal total cycles: 207667200/130 = **1597440**					
W size: 307200	W reuse: 676		I size : 43200		I reuse : 4807.11			O size: **173056**		O reuse: **1200**	

A demonstration: Output loop set (in Figure 3) analysis

loop index	0	1	2	3	5	6	7.1	7.2	7.3	8	10	11
loop level	L0: MAC	L1: Inner-PE Register File								L2: Global Buffer		L3: DRAM
loop	MAC	K 8	C 2	FX 5	OX 2	C 2	FYu 5	OYu 13	OYu 2	OX 13	C 12	K 32
loop relevance	**r 1**	**r 8**	**ir 2**	**ir 5**	**r 2**	**ir 2**	**iru 5**	**ru 13**	**ru 2**	**r 13**	**ir 12**	**r 32**
Data size (elem)	1	16 (inside of each PE), 416 (inside of total PEs)								5408		173056
MAC operation	1	1600 (inside of each PE), 41600 (inside of total PEs)								6489600		207667200
Data reuse factor	1	Total: 100 (Spatial: 5, Temporal: 20)								Temporal: 12		1
Data access count	← 207494144 (Total MAC Op - Output Size)								1903616 ←	1903616	0 ←	0
	→ 207667200 (Total MAC Op)				Divide by current-level total data reuse factor (5x20)				2076672 →	2076672	173056 →	173056 (Output Size)
Turnaround cycles	1	320								49920		1597440
Unique unit count	26	26								1		1
Required average mem BW (elem/cc)	← 1 / 1	Partial output flow bidirectionally; Final output flow unidirectionally.							16 / 320 ←	416 / 320	0 ←	0
	→ 1 / 1								16 / 320 →	416 / 320	5408 / 49920 →	5408 / 49920

Fig. 6. A demonstration: extract loop information from Output loop set in Figure 3 based on loop relevance principle.

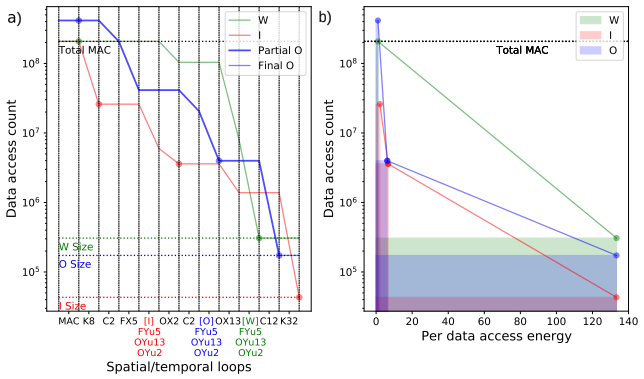


Fig. 7. Visualization of a) the impact of individual loops (in a bottom-up order) on data access count, and b) energy consumed by different memory levels (the area of these blocks indicates energy), using the dataflow example in Figure 3. Note that the highlighted dots in two figures are one-to-one correspondence and are corresponding to the data access at different memory levels for different operands.

ory energy estimation, can also be easily extracted. The first term in the formula, "Operand Size" is how many elements in total W, I, or O has; the second term is how many times each element needs to be accessed repetitively at the current memory level, which equals to the product of the total data reuse factor at current level and all the levels above. Figure 7a visualizes individual loop's impact on the memory access count. The circle markers indicate the boundary of the memory levels, showing the actual number of memory accesses for each memory level for each operand.

10.) Required memory bandwidth is the minimum bandwidth that ensures computation happen fluently without stall. It depends on both dataflow and memory settings. Without double-buffering, writing only happens after a specific data item is fully used, resulting in a small time window. With double buffering, writing can happen all the time (in parallel with data loading), leaving a large writing time window, and thus lowering required instantaneous memory bandwidth. The bandwidth difference between these two cases is the product of all the top 'ir" loop values.

Note that due to the 'pr' loops, some changes are needed for handling the Input. The most important modification are the following two substitutions. One is to correctly handle data size (assuming stride is 1):

$$\prod_{Lmin}^{Li} r \rightarrow \prod_{Lmin}^{Li} r \cdot \Big(\prod_{Lmin}^{Li} pr_1 + \prod_{Lmin}^{Li} pr_1' - 1\Big) \cdot \Big(\prod_{Lmin}^{Li} pr_2 + \prod_{Lmin}^{Li} pr_2' - 1\Big)$$

in which pr_1 (pr_2) and pr_1' (pr_2') are a pr loop pair, like OX and FX. Another substitution is to correctly handle special Input data reuse cases like the "diagonal broadcast" and "FIFO Effect":

$$\text{Total data reuse factor @ Li} \rightarrow \frac{\text{Total MAC Op @ Li(+pr)}}{\text{Total Data Size @ Li(+pr)}}$$

For example, in the "FIFO Effect" setting: $\frac{\text{FX 3}}{\text{OXu 4}}$, the lower-level data reuse factor should equal to $\frac{(3\times4)\text{ MAC Op}}{(3+4-1)\text{ data}} = 2$ instead of $\frac{(1\times4)\text{ MAC Op}}{(1+4-1)\text{ data}} = 1$ by taking the "FIFO effect"-triggering 'pr' loop FX 3 into account.

C. Hardware Cost Integrator

The Hardware Cost Integrator aims at integrating the extracted technology-independent loop information with the technology-dependent characteristics to estimate the final hardware cost and performance, namely energy, throughput, and area.

1.) Area: Area estimation is straightforward, summing up all the used on-chip memory's area, as it is dominant.

2.) Energy: MAC computation energy and memory access energy are taken into account. MAC computation energy is estimated by multiplying total number of MAC operations with average single-MAC-operation energy; memory access energy is calculated by multiplying the memory access count, provided by the Loop Information Extractor, with the corresponding memory per-data-access energy, taking into account the memory size, the potential memory bitwidth mismatch overhead, operand precision, and data stationarity. Figure 7b visualizes this step: the energy consumed by each memory level (L1 register file, L2 global buffer and L3 DRAM) of each operand is visualized by the area of each block, showing that a good dataflow leads to high data access counts at low-cost

memories with low data access counts at high-cost memory. A reliable wire cost model for interconnection energy estimation is planned for future work.

3.) Latency/Throughput: PE array utilization, throughput, and latency are tightly related and can be deduced from each other. A PE array's under-utilization can come from spatial stalls and temporal stalls. Spatial stalls result from mismatch between the spatial unrolling dimensions and the neural network layer dimensions. Temporal stalls mainly come from memory bandwidth bottlenecks during computation.

ZigZag analytically estimates both types of stalls. Spatial stalls are straightforward. Temporal stalls are calculated by comparing the actual memory bandwidth with the required memory bandwidth (derived from Loop Information Extractor), which is tightly coupled to memory type and dataflow.

Figure 9 gives an toy example of extracting required memory bandwidth with two memory scenarios under a memory level with a 'ir' loop on top. In Figure 9a), without double-buffering, writing only happens when one datum is fully used, thus the time window left for data writing is small, while in Figure 9b), with A and B buffer, writing can be totally overlapped with reading, leaving a large time window. More specifically, the required writing memory bandwidth for case a) is $6/24 = 0.25$, for case b) is $6/120 = 0.05$. The ratio between them is exactly the size of top-ir loop.

After getting the required memory bandwidth, the next step is analyzing ideal memory data transfer duration and data transfer period, i.e. understanding how long and how often one memory is working. Then, stalls due to limited memory bandwidth can be calculated by the equation shown and explained in Figure 8.

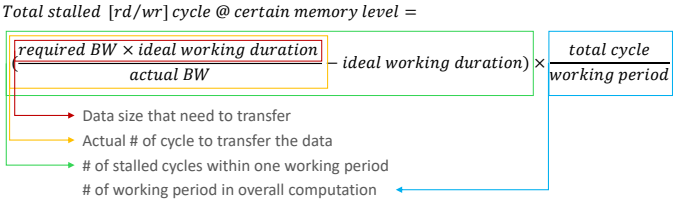


Fig. 8. Stalls due to limited memory bandwidth.

The practical situations are usually more complicated. ZigZag's performance analysis incorporates the following factors: memory bandwidth, memory sharing between W/I/O, memory type (single-port/dual-port, wi/wo double-buffering), memory spatial unrolling, and partial/final sum of output.

Besides estimating latency, another potential of ZigZag's performance analysis is detecting run-time memory gating possibilities. For example, Figure 10a) and b) show two valid memory schedules (with the same required memory bandwidth) to support the smooth computation of the same dataflow schedule (written in 'r'/'ir' loop format). Notice that b) need less memory size than a), which indicates that, theoretically, 60% of the memory size in scheme a) can be gated. We call the minimal required memory size to support certain dataflow schedule performing smoothly without affecting the memory bandwidth requirement as "effective memory size". The memory part that exceeds the "effective memory size" can theoretically be gated to save power without affecting performance. ZigZag provides the effective memory size analysis for each valid mapping point.

IV. Temporal Mapping Generator

The workload is expressed as a set of nested-loops that determine the order of execution of the MAC operations within each PE. Since the operands for the loops are distributed in the memory hierarchy, each loop is mapped on a level at an index order. The order, size and type of the nested-loops determines the *temporal mapping*.

The ZigZag mapper efficiently searches for loop schemes on even and uneven memory hierarchies that present shared and/or non-shared levels between different operands. Thus, it *significantly* increases the design space with respect to previous works, without missing the optimal solution.

By adopting an enhanced loop blocking space representation and using the concept of *roofs* and *virtual levels*, the proposed ZigZag mapper can efficiently support:

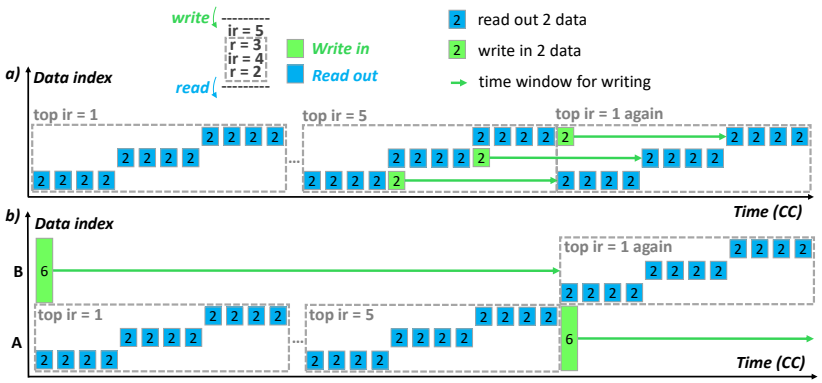
- 2D and 3D convolutional layers, pointwise convolutional layers and fully-connected layers. Depthwise layers can be expressed as a combination of Conv2D and pointwise layers as described in Figure 2;
- Memory hierarchies with memory levels that store separate operands and/or memory levels that are shared with two or three types of operands, so as to allow maximum flexibility in the hardware design space;
- Multiple levels of spatial unrolling;
- Three mapping space exploration methods: exhaustive search, heuristically-optimized search and non-heuristically iterative optimized search.

A. Enhanced Loop Blocking Representation

1) Loop prime factors: The module generates all the valid temporal mappings, that are characterized by their type, size and order of the nested loops. In order to fully explore all possible combinations of schemes, we will refer to *loop prime factors* (LPF) as the smallest sizes in which a loop can be split, or in other words, the atomic blocking sizes that cannot be further divided.

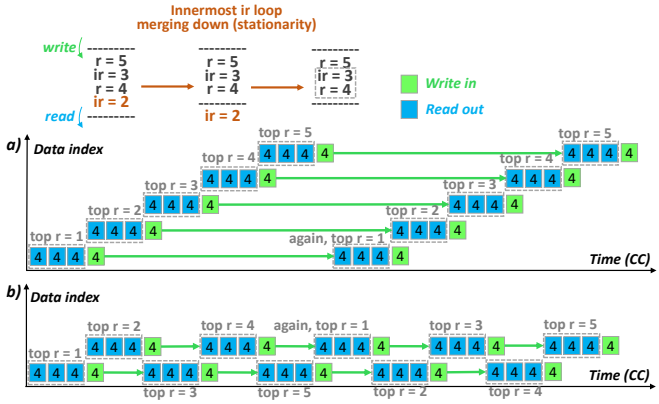
These LPFs correspond to the result of the factorization of the layer dimensions and are the basic blocks of the search algorithm. Starting from the smallest memory level in the hierarchy, the search method proceeds in successive allocations of these prime factors and generates all valid assignment combinations of the LPFs in the given memory hierarchy.

2) Roofs and virtual memory levels: Much of the previously published temporal mapping search methods only dealt with even mappings. In those search methods, every for-loop belonged to the same memory level for inputs, weights and outputs, meaning a considerable restriction of the mapping space. With even hierarchies the mapping process of the blockings purely implies a fast check whether the size of the blocking combination and loop order fits within the available space in the corresponding memory level. In contrast, under uneven hierarchies, a single blocking index can belong to different memory levels for different operands as in Figure 11.



Current-level required Wr BW or Above-level required Rd BW: (Take Weight for example, without considering memory sharing or memory unrolling)	Required memory size
a) total data size/total CC (exclude the top ir) = (3*2)/(3*4*2) = **0.25 element/CC**	**6 elements**
b) total data size/total CC = (3*2)/(5*3*4*2) = **0.05 element/CC**	**12 elements**

Fig. 9. ir-loop-on-top dataflow's implication on required memory bandwidth and memory size with a) non-double-buffering and b) double-buffering. Assuming a) is with one r/w dual-port memory and b) is with two single-port memories. "CC" stands for clock cycle; "BW" stands for bandwidth.



Current-level required Wr BW or Above-level required Rd BW: (Take Weight for example, without considering memory sharing or memory unrolling)	Required memory size
a) total data size/total CC = (5*4)/(5*3*4) = **0.33 element/CC**	**20 elements**
b) subset data size/subset CC (exclude the top r) = (4)/(3*4) = **0.33 element/CC**	**8 elements**

Fig. 10. r-loop-on-top dataflow's implication on required memory bandwidth and memory size. Assuming both a) and b) are with one r/w dual-port memory.

loop index	Weight			Input			Output		
8	DRAM	K	4	DRAM	K	4	DRAM	K	4
7		C	24		C	24		C	24
6		OX	2	Global	OX	2	Global	OX	2
5		K	4	Buffer	K	4	Buffer	K	4
4		OX	13		OX	13		OX	13
3		K	16		K	16		K	16
2	Inner-PE	C	2	Inner-PE	C	2	Inner-PE	C	2
1	Reg File	FX	5	Reg File	FX	5	Reg File	FX	5
0	MAC Level			MAC Level			MAC Level		

	Weight			Input			Output		
DRAM	K	32			K	32	DRAM	K	32
	C	12		Global	C	12	Global	C	12
	OX	13		Buffer	OX	13	Buffer	OX	13
	C	2			C	2		C	2
	OX	2			OX	2		OX	2
	FX	5			FX	5		FX	5
Inner-PE	C	2		Inner-PE	C	2	Inner-PE	C	2
Reg File	K	8		Reg File	K	8	Reg File	K	8
MAC Level				MAC Level			MAC Level		

Fig. 11. The best even temporal mapping of AlexNet CONV2 on Eyeriss (left) compared to the best mapping globally, which is uneven (right). The uneven one can achieve 20% energy saving with the same spatial mapping, "FYu|OYu|OYu 5|13|2" at Inner-PE Register File level (not shown in the figure).

To handle these scenarios, ZigZag introduces *virtual memory levels* and the *roof* variable.

The assignment process is guided by the *roof* variable, which is a tuple defined for each operand, containing 1) the memory level index where the LPFs are being assigned and 2) the maximum amount of relevant prime factors that can still be assigned in the available space in the specified memory level. The roof is initialized with the smallest memory level in the hierarchy for each operand since the assignment begins from the innermost level of the hierarchy. Its value is updated after each assignment by dividing the available space by the product of the size of the relevant loops allocated.

A single assignment step is described in Figure 12: the memory hierarchy is the same as Eyeriss and the workload is CONV2 of AlexNet. The figure contains a partial scheme with some LPFs already assigned in the innermost memory levels. The reported partial scheme is one of many other possible ones that have been obtained with previous LPF assignment steps. In Figure 12, before the assignment of the LPFs, the second level of input memory hierarchy (at index 1) has a storage capacity of 884736 bits, as described in Figure 3 or 884736/16 = 55296 blocks that can be stored with precision 16 bit. The prime factors that this level holds are those already assigned and relevant in the levels below $((\texttt{FX},5),(\texttt{C},2))$ plus those relative to the relevant spatial unrolling below $((\texttt{FYu},5),(\texttt{OYu},13),(\texttt{OYu},2))$ that combined correspond to $5 \times 2 \times (5 + 26 - 1) = 300$ blocks. Therefore, the second level can still store $55296/300 = 184.32 \sim 184$ blocks. The

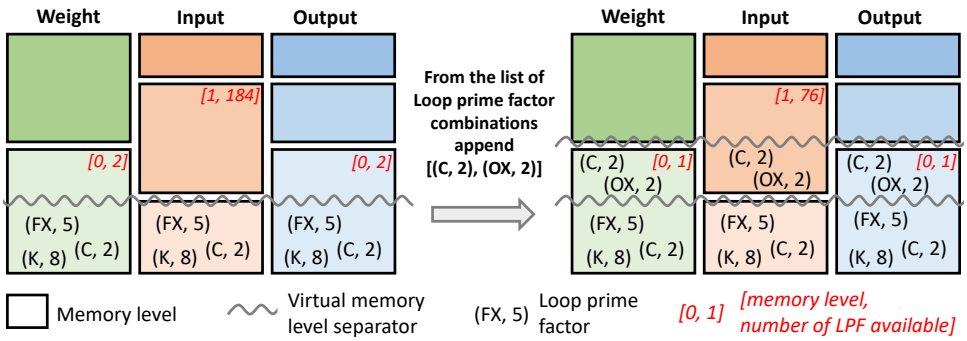


Fig. 12. The LPF assignment step in a partial scheme and the successive update of the roof variable. The roof values are those in red: they are defined for each operand and updated after each LPF allocation.

Algorithm 1: Blocking scheme generator

```
Initialize EmptyScheme.roof and EmptyScheme.LPFList
Append EmptyScheme to PartialSchemes
while LPFList is not empty for all PartialSchemes do
    for Ps in PartialSchemes do
        for k in range(0, len(Ps.LPFList)) do
            CombList ← combinations(Ps.LPFList, k)
            for LPFComb in CombList do
                if fitComb(LPFComb, Ps.roof) then
                    Update Ps with LPFComb
                    Remove LPFComb from LPFList
                    Append Ps with LPFList to
                     PartialSchemes
                end
            end
        end
    end
end
```

roof for the input in this scheme is thus `[1, 184]`.

In the described case, the LPF combination of `(C,2)` and `(OX,2)` is one of the many combinations found to be fitting within the roof limits and is appended to the partial scheme. After its allocation the available space at the same level becomes $55296/[(5+2-1)\times 2\times 2\times(5+26-1)] = 76$ blocks and thus the roof will be updated to `[1, 76]`.

When no LPF combination is found to fit within the roof of all operands then the smallest roof, identified as either the one with the smallest memory index or the one with the least space available, jumps to the next level available in the hierarchy. After that, a new search for the fitting combinations with the updated roof restarts.

Subsequent to each assignment is the placement of a virtual memory level separator, which creates a fictive memory level in which all operands have equal sets of LPFs, as in Figure 12. When all LPFs are assigned, the memory hierarchy is organized in virtual memory levels, within which its LPFs can be permuted. All the possible permutations are generated and sent for evaluation by the hardware cost model.

3) Shared and non-shared memory levels: A shared memory level in the hierarchy is a level in which multiple operand types can be stored, and different operands can occupy different portions of the space available in the level depending on the blocking scheme assigned. The presence of shared memory levels greatly increases the amount of blocking scheme combinations since they act as levels which have a flexible upper bound for the space available for each operand: having fixed the minimum utilization rate of the shared level (usually 70%), depending on the blockings already assigned, different operands can occupy a larger or smaller chunk of the storage space.

B. Exhaustive Search

Depending on the complexity of the hierarchy and the workload, this search method generates all possible schemes through loop blocking and loop reordering in an exhaustive way. It can take multiple hours to run the search and evaluate millions of valid mappings for a single layer, of which only a few are optimal ones. Speed-up techniques are required to explore the mapping space more efficiently, resulting in the heuristic and the iterative search introduced underneath.

C. Heuristic Search Based On Data Reuse and Stationarity

Once LPF assignment is completed, the data reuse for each operand and level can be extracted. If a particular combination of loop prime factors causes the data-reuse to be equal to 1 at a specific level in the hierarchy, it follows that that level is unnecessary since it causes useless memory accesses.

Consequently, the heuristic search discards all mappings with data reuse values equal to 1 for intermediate levels in the hierarchy (excluding the innermost and outermost ones). It is important to note that this rule does not hold for Input data, as even with data reuse equal to 1, this level may exhibit the FIFO effect and be optimal.

Successive to this solution reduction step, the permutation of the LPFs is carried out again within the virtual memory levels to generate only those schedules that maximize stationarity for each operand (W/I/O), in order to avoid trying out all the permutations.

D. Iterative Search Based On Early-Stage Cost Evaluation

The last search strategy proposed explores the mapping space in an *iterative* way, instead of generating an exhaustive

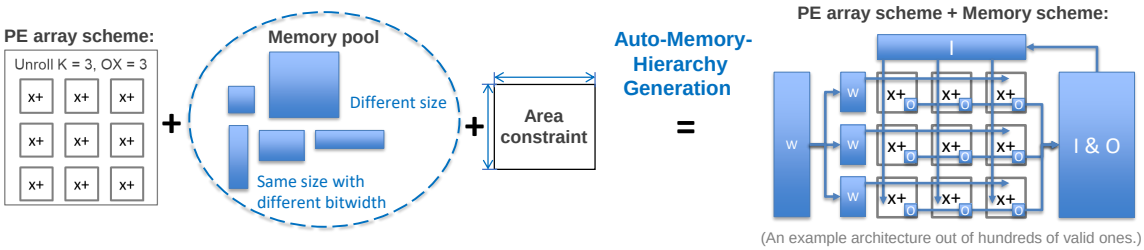


Fig. 13. Memory hierarchy generator overview.

list of blocking scheme combinations first and possibly pruning away the sub-optimal ones. Starting from the innermost level of the hierarchy, an iteration step consists in finding the set of LPFs that causes the largest amount of energy savings. After each iteration the best virtual memory level found is stacked upon those previously found ones.

Since this scheme analyzes partial schemes before converging to an optimal point and ignores the influence of the upper levels in the hierarchy when making a decision at the lower levels, it might reach a sub-optimal point in the temporal mapping space. We will show in the case studies later that the energy-overhead is however almost always within 5% of the cost of the optimal mapping. However, the speed-up with respect to the other methods (up to 10×) and the much smaller memory footprint (˜100s of mappings to be stored at each iteration step compared to the millions of the heuristic search) may be worth the trade-off for many DSEs.

V. Architecture Generator

The performance of an embedded system is determined by the joint effect of the execution schedule combined with the hardware architecture, as observed in [16]. The search for the optimal design should therefore not ignore the influence of the latter element, whose most influential component is the memory hierarchy.

The ZigZag architecture generator aims to autonomously generate all valid memory hierarchies in a search space constrained by area, PE array dimensions and spatial unrolling scheme. It draws the memories to build the hierarchy from a pool of available memory instances with different memory sizes and memory bandwidths. For each feasible memory hierarchy that fits the area constraint, the optimum memory bandwidth is selected based on the theory presented in Section III-C. The addition of this generator effectively adds another dimension in the design space, yet, enables the designer to find the best point in the three-dimensional space described by area, energy consumption and throughput.

A. Comprehensive Memory Pool Description

Each memory in the pool is defined by its storage size expressed in bits, its cost of access expressed in pJ for read and for write, and its area in μm^2.

Given that the framework is also able to determine what would be the required memory bandwidth, so as to maximize the throughput of an architecture, each of the parameters of the memories in the pool are described for several distinct bandwidths as well.

TABLE IV
Sample of 64 Byte RF in the pool for an 8 × 8 PE array. Tuple values = [*read*, *write*] conditions.

Size [bit]	2048		
Area [μm^2]	4740.06	4825.56	7457.82
Access cost [pJ]	[0.88, 0.98]	[0.99, 1.39]	[2.52, 3.49]
Mem bw [bit]	[8, 8]	[16, 16]	[64, 64]
Unroll	1, 8, 64		

The cost parameters (area, access energy for each bandwidth) have to be defined as input and are technology dependent: their accurate definition is vital to obtain the optimal design point. For running our estimation the CACTI 7.0 tool has been deployed at 65 nm, but these values can be fixed by the user before each simulation run so as to mirror the technology that is actually being deployed. Each memory in the pool is characterized as in Table IV.

The unroll parameter specifies the amount of times the memory level is replicated in the architecture.

B. Memory Hierarchy Generation

Figure 13 gives an overview of the function of the memory hierarchy generator.

The generation of the set of valid memory schemes consists of three successive stages, respectively the *fitting of the memories*, the *operand assignment* and the *bandwidth optimization*.

In the first stage the memory pool is firstly extended to include the unrolled version of the single memories as well, so as to have the possibility to have memory levels present in all PEs of the array or unrolled along a single dimension of the array. Subsequently all the fitting combinations with repetition (with max repetition set to 3 as the number of operands) of the memory elements from this enhanced pool are generated and assigned to an operand or to a set of operands as in the case of shared memory levels.

The output of this stage is a list of valid memory hierarchies which are sequentially fed to the schedule generator, which in turn finds for each the optimal temporal mapping and its required bandwidth by means of the hardware cost model. When all hierarchies are analyzed, the optimal memory hierarchy and its optimal temporal mapping for a specific layer in a network will be identified.

VI. Validation

The hardware cost model and the mapping engine are validated with three methodologies: a.) against published taped-out chips measured results; b.) against in-house post-synthesis

extracted energy and performance data; c.) against other DNN accelerator DSE frameworks.

Firstly, we model the dataflow and hardware architectures of both Eyeriss [2] and ENVISION [1] and compare the estimated energy (left bars) with their reported values (right bars), as depicted in Figure 14. The resulting energy values, normalized with respect to a single MAC cost, are shown for full precision operation without voltage scaling or sparsity reduction. The estimated values are within an acceptable 5%, resp. 7.5% error margin.

Secondly, the validation of the energy as well as the performance model is performed against a complete in-house accelerator at RTL level, shown in Figure 15. A maximum error of 6% of energy and 9% of PE array utilization are achieved.

Finally, the validation of the cost model as well as the temporal mapping generator is carried out against two SotA frameworks: Timeloop [16] + Accelergy [14], resp. Interstellar [12], as shown in Figure 16. For Timeloop + Accelergy, the ResNet34 [21] convolutional layers are first mapped on the Eyeriss hardware architecture. Subsequently we estimate the energy cost of this (suboptimal) mapping through Timeloop + Accelergy ('TL' in Figure 16 left), and ZigZag ('TL on ZZ'), matching within 10%. Yet, when we let our temporal mapping generator optimize the scheduling for the same architecture and workload, more optimal design points are found, which can lead to up to 20% energy savings ('ZZ'). The optimal mapping exploits uneven mapping, and can not be validated back with Timeloop, since it cannot be represented by Timeloop's limited design representation. Note that a lot of ReNet34 layers have the same dimension size, thus we only pick all the non-repetitive layers for validation.

A similar validation method is applied to Interstellar on a hardware template with three all-shared memory levels. Several pointwise layers in MobileNet V1 [22] are used for testing, since their framework cannot handle the 'pr' loop pair data reuse accurately. Note that in this two-step experiment (firstly matching energy with Interstellar's best schedule, and then searching for an better one in our enlarged design space), we ignored the same energy contributions that Interstellar ignored, such as distinguishing psums and final sums, separating cost of memory writing from reading, etc. The result shows that an total energy matching with only 3% error is achieved, and our uneven mapping scheme outperforms its best even mapping scheme by up to 33% concerning energy.

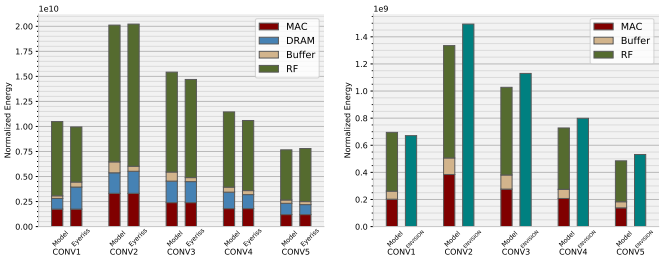


Fig. 14. Model validation of AlexNet CONV layers on Eyeriss (left) and ENVISION (right).

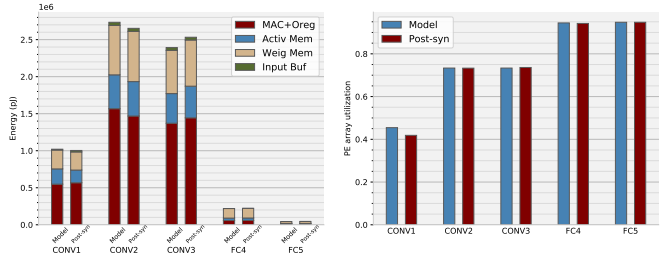


Fig. 15. Model validation against an in-house accelerator's post-synthesis results with a voice recognition workload on energy (left) and PE array utilization (right).

VII. Case Studies

To better understand the vast design space and show the strength of ZigZag, three case studies from different design abstraction levels are conducted.

A. Case Study 1: Impact of Scheduling

The cost estimator and temporal mapping generator of ZigZag are used to assess the impact of scheduling on both energy and throughput. This is assessed for AlexNet convolutional layer 2 on an Eyeriss-like architecture. A memory bandwidth of 16 bit/cycle is assumed for RF and 64 bit/cycle for GLB. The results in Figure 17 shows there is an up to $4.7\times$ energy variance and an up to $8\times$ throughput variance across temporal schedules.

A striking observation that can be made is how limited the space of exploration is if only even mappings are considered: the number of uneven mappings is thousands of times larger than the number of the even ones and the uneven ones can reach optimal design points that would be otherwise not achievable. For this particular case study an improvement of 25% of the energy value can be obtained with respect to the best even mapping. Similarly, as is the case for the validation tests run in the previous section, comparable improvements are achieved for different architectures and workloads as well.

Next, the three ZigZag search engines (Section IV) are compared in terms of their searching efficiency. Figure 18 visualizes their search procedures and obtained results. The figure contains one line for each represented schedule. For each schedule, this line depicts the energy spent on memory accesses assuming all lower level loops are scheduled, and all upper level loops are assigned to DRAM (Figure 7a converted to energy). The rightmost point of every curve is the actual energy of the completely scheduled workload.

In Figure 18, the grey curves are thousands of randomly-sampled valid schedules the tool found in exhaustive search, while the orange curves give results from the heuristic search. The iterative search only has 1 trajectory, as it only refines a single solution. The bold curves mark the trajectory of the minimal-energy schedules found by each strategy, plus the schedule reported in Eyeriss paper. Notice that the best schedule found by exhaustive search and heuristic search overlap, meaning that the heuristic search can equally well locate the global optimum schedule as the exhaustive search does. Iterative search resulted in another schedule, which

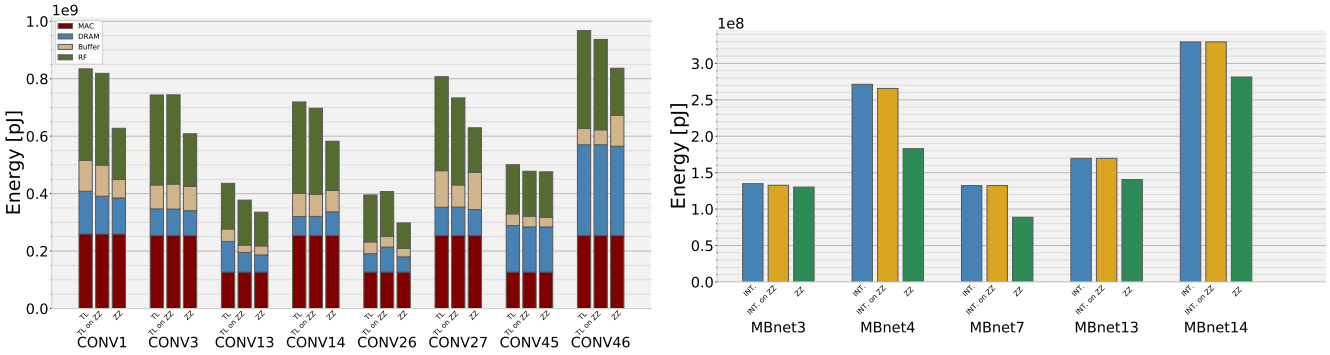


Fig. 16. Model validation against Timeloop+Accelergy (left) and Interstellar (right). In each group of three bars, the left/middle/right bars are respectively the best schedules found & evaluated by SotAs, the best schedule found by SotAs & evaluated by ZigZag, and the best schedule found & evaluated by ZigZag.

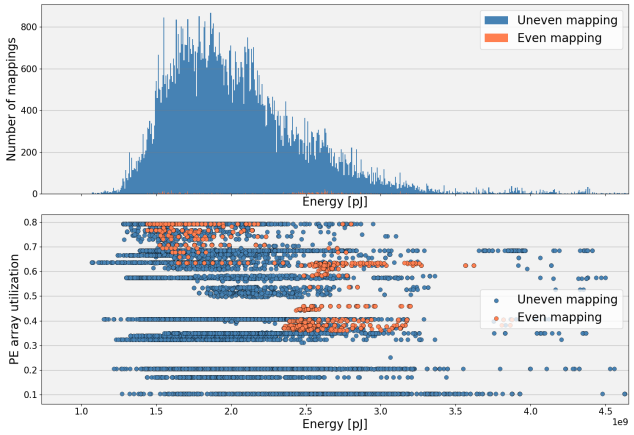


Fig. 17. Schedule's impact on energy and throughput with even/uneven blocking on AlexNet CONV2 mapped on Eyeriss (access energies derived from CACTI7).

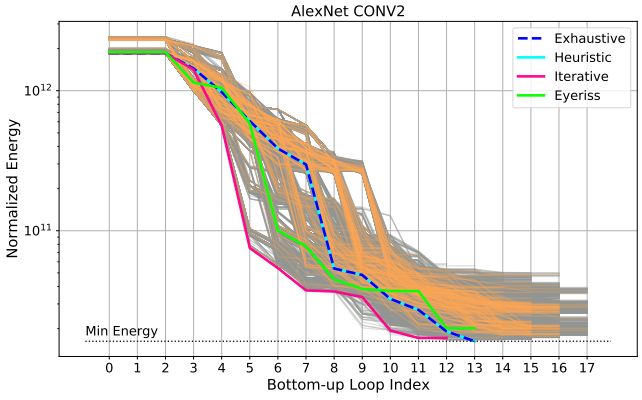


Fig. 18. Visualization of individual loops impact on energy saving and three searching strategies' trajectory.

is slightly (5.5%) more energy consuming than the global optimum. The Eyeriss schedule is 23.8% worse than the global optimum. Table V gives an overall comparison of using these three searching strategies on locating the best schedule for AlexNet conv. layer 1-5. It shows that heuristic search can bring a 2.5× speedup without losing optimality, while iterative search brings a 7.5× speedup with a 1.6% energy penalty on average.

TABLE V
COMPARISON ON THREE SEARCHING STRATEGIES

AlexNet CONV1-5	**Exhaustive Search**	**Heuristic Search**	**Iterative Search**
Total Number of Valid Schedules	4887334	1444608	Partial: 3810973 Final: 7168
Relative Number of Schedules	100%	30%	Partial: 78% Final: 0.15%
Elapsed Time (sec)	13228.2	5330.4	1760.1
Speedup	×1	×2.5	×7.5
Minimum Energy	1	1	1.016

TABLE VI
CASE STUDY 2 INPUT PARAMETERS

PE array size	8 × 8
Spatial unrolling	OX — K
Memory pool	64 Byte, 1KByte, 4KByte, 16KByte, 128 KByte, 2 MByte @ 65nm
Workloads	MobileNet V2 - L13, L15, L39, L46
Temporal mapping	Optimal with heuristic search

B. Case Study 2: Workload and Memory Hierarchy

The best design co-optimizes the dataflow schedule and the hardware architecture. Here, every workload (e.g. neural network layer) would have a different optimal memory hierarchy and dataflow schedule.

Yet, in reality, in most designs this is impossible as the memory levels are hardwired on chip. This raises the question of whether the flexibility overhead from reconfigurable memories with a network-on-chip (able to dynamically change the memory hierarchy between layers) is amortized by its benefits. To evaluate this, we deployed the architecture generator and the temporal mapping generator on different layers from MobileNetV2, namely the layers 13, 15, 39 and 46. Each layer is constrained to the same PE array with the same spatial unrolling, and the memory hierarchy and temporal mapping are jointly-optimized for minimal energy consumption. The input parameters are listed in Table VI.

Table VII summarizes the result of this study. It suggests that having a flexible hierarchy may be worth the trade-off if the energy cost overhead of having a Network-on-Chip is within the 30% of the total inference cost of the layer. Figure 19 visualizes the design space targeting on one single layer, showing the energy-performance-area tradeoff between

TABLE VII
ESTIMATED ENERGY FOR DIFFERENT WORKLOADS AND THEIR OPTIMAL ARCHITECTURES.

	Optimal memory hierarchy and its spatial unrolling	Run L13	Run L15	Run L39	Run L46	**Total**
L13 Arc.	W: [16777216], I: [32768, 16777216], O: [512, 32768, 16777216] W: [[(K, 8), (OX, 7)],[]], I: [[(OX, 7)], [(K, 8)],[]], O: [[],[(OX, 7)],[(K, 8)],[]]	27.57 μJ	60.71 μJ	59.47 μJ	44.79 μJ	192.54 μJ
L15 Arc.	W: [16777216], I: [512, 8192, 16777216], O: [512, 32768, 16777216] W: [[(K, 8), (OX, 7)],[]], I: [[(OX, 7)], [(K, 8)],[]], O: [[],[(OX, 7)],[(K, 8)],[]]	58.79 μJ	28.37 μJ	60.22 μJ	41.69 μJ	189.07 μJ
L39 Arc.	W: [32768, 16777216], I: [16777216], O: [512, 32768, 16777216] W: [[(K, 8), (OX, 7)], [], []], I: [[(K, 8), (OX, 7)], []], O: [[], [(K, 8)], [(OX, 7)], []]	41.69 μJ	40.31 μJ	49.51 μJ	45.09 μJ	176.6 μJ
L46 Arc.	W: [16777216], I: [32768, 16777216], O: [512, 32768, 16777216] W: [[(K, 8), (OX, 7)], []], I: [[(K, 8), (OX, 7)], [], []], O: [[], [(OX, 7)], [(K, 8)], []]	29.25 μJ	64.15 μJ	51.34 μJ	34.37 μJ	179.11μJ
Flexible architecture						139.82 μJ

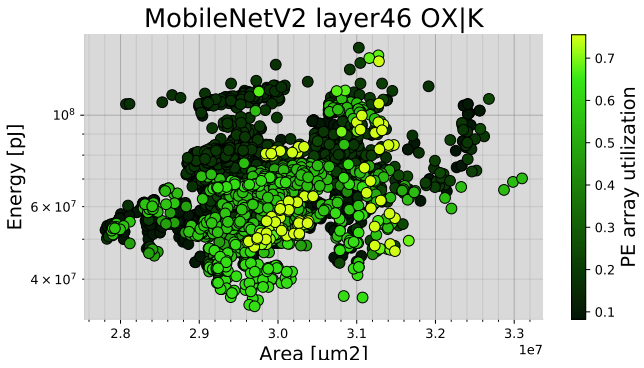


Fig. 19. Design points identified by the framework for L46 of MobileNetV2. Each dot corresponds to a different memory hierarchy solution.

hundreds of valid design points.

C. Case Study 3: Spatial Unrolling and Memory Hierarchy

Another degree of freedom in the design space to assess is the PE array's spatial unrolling. Previous works [12] stated that the spatial unrolling has a very limited effect on the energy consumption as long as the PE array is fully mapped.

Figure 20 shows the energy consumption of two different layers of MobileNetV2 for several spatial unrollings and memory hierarchies. The number after operands (W/I/O) indicates memory level, e.g. W0 means the 0th (innermost) memory level (usually register file) of Weight. The leftmost two memory hierarchies have all their levels shared among the operands, the rightmost two are hierarchies with different memory levels for each operand. The results indicate that spatial unrolling has limited energy impact only when memory levels are shared between all operands. It is because having shared memories softens the constraint on the memory utilization and makes the occupying size of each operand (W/I/O) at each memory level flexible, thus enabling a much larger number of temporal mappings, making it possible for the temporal mapping to adapt itself to the spatial unrolling. In other words, temporal mapping can compensate the unbalanced data reuse distribution in spatial unrolling among W/I/O operands with all-shared memory hierarchies.

Yet, the rightmost two hierarchies have no memory sharing among the different operands and have widely distributed energy costs: in this scenario such compensation is not possible, as the Temporal Mapping Generator has less degrees of freedom to arrange the schedule and balance the temporal and spatial data reuse. The spatial unrolling scheme here impacts energy efficiency up to 5×.

VIII. CONCLUSION

This paper presents ZigZag, a memory-centric rapid design space exploration framework for DNN accelerators.

Three modules cooperate in synergy to enable the exploration of a much broader space of solutions with respect to the SotAs. Firstly, the Architecture Generator is capable of generating all valid memory hierarchies (balanced/unbalanced, shared/separate) given a set of high-level hardware constraints. Secondly the Temporal Mapping Generator can rapidly locate the optimal schedule (even/uneven) by means of innovative searching methods for any type of memory hierarchy provided by the Architecture Generator. Thirdly, with the memory-centric dataflow representation and the Loop Relevance Principle, the Hardware Cost Estimator can analytically calculate energy and throughput for the schedules generated by the Temporal Mapping Generator.

Three case studies disclose the vast DNN accelerator design space from different perspectives. The first experiment shows the importance of adopting an optimal schedule when mapping the algorithm onto the hardware since it has huge impact on both energy and performance and uneven mappings opens up the searching space and leads to find better design points. The second experiment highlights that different workloads lead each to their own optimum memory hierarchies; it also assesses whether a Network-on-Chip that enables configurable memory bypassing and memory operand re-assigning is worth the implementation so as to enable the mapping of each workload on its own optimal memory scheme. The third experiment partly disproves the conclusion drawn by Yang. et al [12] that spatial unrolling is unimportant as long as the PE array is fully mapped. Our results shows that this conclusion can only hold for memory hierarchies with all shared levels, in which the operand with less spatial data reuse can be well compensated by its temporal data reuse in local storage; in memory hierarchies with not all the levels shared, the choice of the spatial unrolling can instead greatly affect the overall cost.

In conclusion, we showed the great capabilities and the uniqueness of ZigZag in exploring the design space of DNN accelerator.

The research team is continuing building and polishing ZigZag. At the same time, we have open-sourced this project at https://github.com/ZigZag-Project/zigzag and welcome comments and contributions from the community.

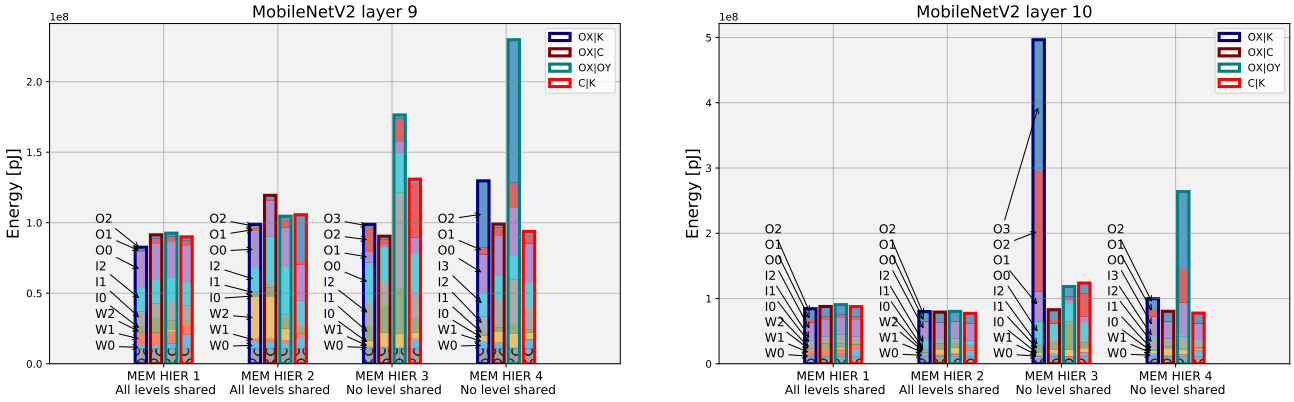


Fig. 20. Influence of spatial unrolling on Shared vs Not-shared memory hierarchies.

ACKNOWLEDGMENTS

This research received funding from the Flemish Government (AI Research Program) and the Fund For Scientific Research Flanders (FWO-Vlaanderen).

REFERENCES

[1] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "14.5 envision: A 0.26-to-10tops/w subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28nm fdsoi," in *2017 IEEE International Solid-State Circuits Conference (ISSCC)*, 2017, pp. 246–247.

[2] Y. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[3] B. Moons, D. Bankman, L. Yang, B. Murmann, and M. Verhelst, "Binareye: An always-on energy-accuracy-scalable binary cnn processor with all memory on chip in 28nm cmos," in *2018 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 2018, pp. 1–4.

[4] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[5] J. S. Giraldo and M. Verhelst, "Laika: A 5uw programmable lstm accelerator for always-on keyword spotting in 65nm cmos," in *ESSCIRC 2018-IEEE 44th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2018, pp. 166–169.

[6] D. Shin, J. Lee, J. Lee, J. Lee, and H. Yoo, "Dnpu: An energy-efficient deep-learning processor with heterogeneous multi-core architecture," *IEEE Micro*, vol. 38, no. 5, pp. 85–93, 2018.

[7] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[8] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Computer Architecture News*, vol. 42, no. 1, pp. 269–284, 2014.

[9] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "Shidiannao: Shifting vision processing closer to the sensor," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 2015, pp. 92–104.

[10] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 1–12.

[11] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 27–40, 2017.

[12] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz, "Interstellar: Using halides scheduling language to analyze dnn accelerators," p. 369383, 2020. [Online]. Available: https://doi.org/10.1145/3373376.3378514

[13] S. L. Xi, Y. Yao, K. Bhardwaj, P. Whatmough, G.-Y. Wei, and D. Brooks, "Smaug: End-to-end full-stack simulation infrastructure for deep learning workloads," 2019.

[14] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.

[15] A. Burrello, F. Conti, A. Garofalo, D. Rossi, and L. Benini, "Work-in-progress: Dory: Lightweight memory hierarchy management for deep nn inference on iot endnodes," in *2019 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2019, pp. 1–2.

[16] A. Parashar, P. Raina, Y. S. Shao, Y. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[17] S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava, "Dmazerunner: Executing perfectly nested loops on dataflow accelerators," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3358198

[18] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 52. New York, NY, USA: Association for Computing Machinery, 2019, p. 754768. [Online]. Available: https://doi.org/10.1145/3352460.3358252

[19] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. Emer, S. W. Keckler, and B. Khailany, "Magnet: A modular accelerator generator for neural networks," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.

[20] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105. [Online]. Available: http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv e-prints*, p. arXiv:1512.03385, Dec. 2015.

[22] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv e-prints*, p. arXiv:1704.04861, Apr. 2017.